

Object-Relational Mapping

Parte 3 - Manipulação de Entidades

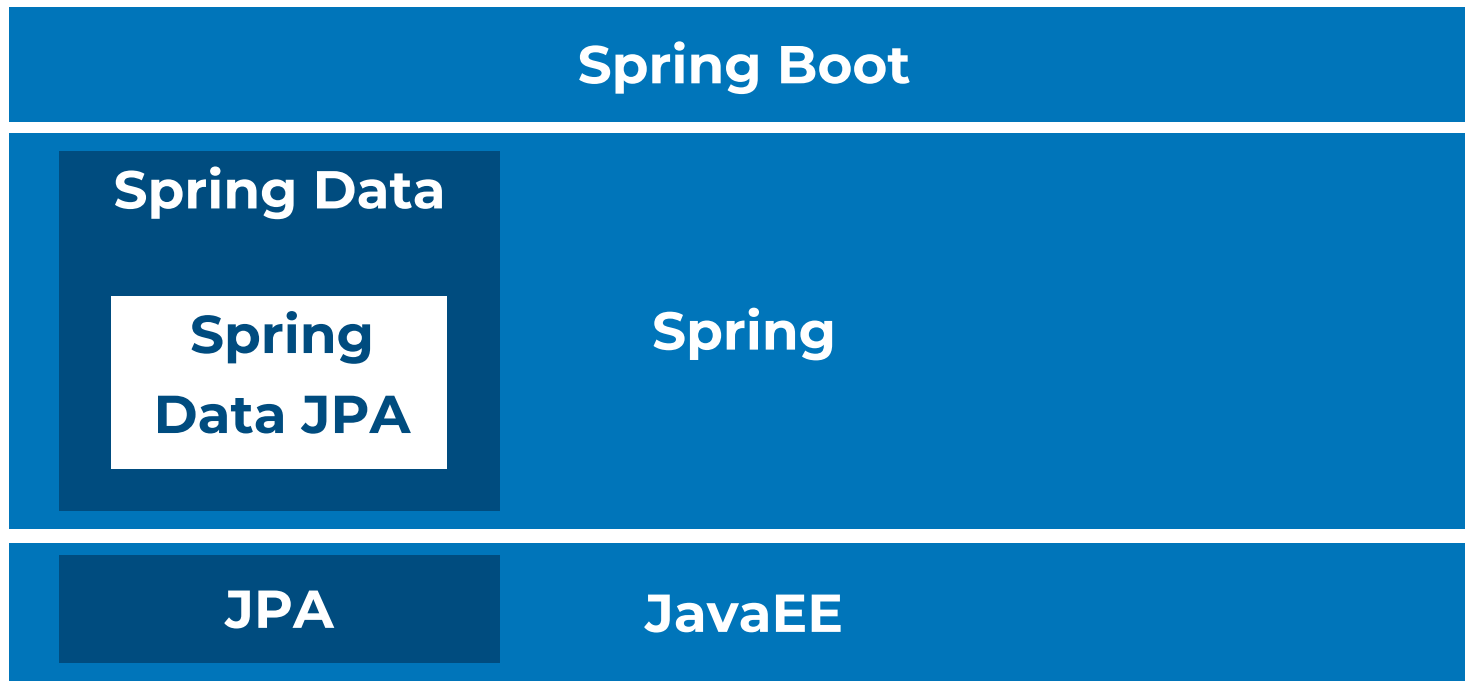
Perspectiva Estática

- Anotações sobre Classes e Atributos.
- Atributos simples, temporais, compostos
- Relações 1-1, 1-N e N-N
- Herança

Perspectiva Dinâmica

- Como persistir instâncias de entidades?
- Como carregar dados da BD?
- Como definir transacções?

Spring Boot e JPA



Repositório em Spring Data

- **Padrão Repository**

- Fornece um objecto que é responsável por encontrar e guardar um tipo de objectos.
- Toma o papel do catálogo que viram em DCO, mas com persistência

- O Repository do Spring:

- Fornece métodos de pesquisa
- Fornece utilitários para paginação e ordenamento.
- Fornece o método save (guarda novas instâncias, ou alterações em já existentes)
- Integra com controladores MVC (próximas semanas)

Repositório em Spring Data

```
@Repository
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    Customer findByVatNumber(String vat);
}
```

Repositório em Spring Data

Anotação necessária para identificar como repositório

```
@Repository  
public interface CustomerRepository extends JpaRepository<Customer, Long> {  
    Customer findByVatNumber(String vat);  
}
```

Utiliza **query derivation** para realizar a busca (através do nome do método relacionado)

Mais sobre isso quando falarmos de Injeção de Dependências!

Obter informação - Repositório

```
interface TutorialRepository {
```

```
    @Query(value = "SELECT t.* FROM tutorials t", nativeQuery = true)  
    List<Tutorial> findAllNative();
```

```
    @Query(value = "SELECT t.* FROM tutorials t WHERE t.published=true",  
            nativeQuery = true)  
    List<Tutorial> findByPublishedNative();
```

```
    @Query("SELECT t FROM Tutorial t")  
    List<Tutorial> findAll();
```

```
    @Query("SELECT t FROM Tutorial t WHERE t.published=true")  
    List<Tutorial> findByPublished();
```

```
}
```

SQL

JPA



person | Enter a SQL expression

	123 id	ABC name
1	1	Serafim
2	2	Zacarias
3	3	Mestre André
4	4	Josefina

my_user | Enter a SQL expression to filter results (use Ct

	123 id	ABC password	ABC username
1	1	benfica	Serafim
2	3	benfica	Mestre André
3	4	benfica	Josefina

admin_user | Enter a SQL expression

	123 id
1	3

editor | Enter a SQL expression

	123 level	123 id
1	0	4

regular_user | Enter a SQL expression to filter

	123 number_of_posts	123 id
1	1	1

-level

PermissionLevel
- HIGH - LOW - MEDIUM
<code>valueOf(name : String) : PermissionLevel</code> <code>values() : PermissionLevel[]</code>

Obter informação - Repositório

```
interface EditorRepository {
```

SQL

```
@Query(value = ""  
SELECT p.id, p.name, u.username, e.level  
FROM Person p  
JOIN my_user u ON p.id = u.id  
JOIN editor e ON p.id = e.id  
WHERE e.level = 2;""", nativeQuery = true)  
List<Editor> findEditorByLevelTwoNative();
```

JPQL

```
@Query("""  
SELECT e  
FROM Editor e  
WHERE e.level = 2")  
List<Editor> findEditorByLevelTwo();
```

```
}
```

Jakarta Persistence Query Language

- Linguagem universal que compila para vários dialectos SQL
- Resolve o problema de expor o SQL à camada de negócio
- É uma extensão do SQL com um sabor a OOP
- É codificada por Strings via a **JPA Criteria API**

JPQL Criteria API

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Item> cr = cb.createQuery(Item.class);
Root<Item> root = cr.from(Item.class);

cr.select(root);

cr.select(root).where(cb.gt(root.get("itemPrice"), 1000));

cr.select(root).where(cb.between(root.get("itemPrice"), 100, 200));

cr.select(root).where(root.get("itemName").in("Skate Board", "Paint", "Glue"));

cr.orderBy(cb.asc(root.get("itemName")), cb.desc(root.get("itemPrice")));
```

- **Padrão Query Object**

- Permite criar um query dinamicamente, traduzindo classes e atributos de Java para código SQL.

Java vs JPQL

- É possível carregar os objectos todos, filtrar em Java, e fazer alterações.
- Mas é mais eficiente filtrar logo em JPQL/SQL.
- E ainda mais eficiente filtrar e **alterar** logo em JPQL/SQL.
- Evita-se criação de objectos, alocação de memória, cópia de dados, etc...

Updates

```
@Modifying
@Query("update User u set u.active = false where u.lastLoginDate < :date")
void deactivateUsersNotLoggedInSince(@Param("date") LocalDate date);
```

```
CriteriaUpdate<Item> criteriaUpdate = cb.createCriteriaUpdate(Item.class);
Root<Item> root = criteriaUpdate.from(Item.class);
criteriaUpdate.set("itemPrice", newPrice);
criteriaUpdate.where(cb.equal(root.get("itemPrice"), oldPrice));
```

```
Transaction transaction = session.beginTransaction();
session.createQuery(criteriaUpdate).executeUpdate();
transaction.commit();
```

Deletes

```
@Modifying  
@Query("delete User u where u.active = false")  
int deleteDeactivatedUsers();
```

```
CriteriaDelete<Item> criteriaDelete = cb.createCriteriaDelete(Item.class);  
Root<Item> root = criteriaDelete.from(Item.class);  
criteriaDelete.where(cb.greaterThan(root.get("itemPrice"), targetPrice));
```

```
Transaction transaction = session.beginTransaction();  
session.createQuery(criteriaDelete).executeUpdate();  
transaction.commit();
```


Transacções

- Casos de uso complexos podem fazer várias alterações à BD
 - Retirar de Stock, Adicionar à compra
 - Semânticas ACID: outras operações não alteram os nossos objectos enquanto estamos a trabalhar neles.
- Solução: **Padrão *Unit of Work***
 - Ter um objecto que representa o que vai ser alterado.
 - Mantém uma lista de objectos afectados/*managed* (registra novos objectos, objectos alterados ou objectos que devem ser apagados)
 - Permite fazer commit (e rollback) das alterações todas.
 - **EntityManager** no JPA

Identity Map

- E se dois Entity Managers carregarem a mesma linha?
- Cada uma podia fazer alterações que iam fazer override à outra!
- **Solução: Identity Map**
 - Existe um mapa de objectos que tem como chave a sua PK.
 - Sempre que carregamos um objecto, confirma-se se existe nesse mapa primeiro, e reusa a cópia pré-existente.
 - No JPA funciona apenas dentro de uma unit-of-work!

EntityManager

- **EntityManagerFactory** - cria entity managers numa unidade de persistência (BD).
- **EntityManager** - gere entidades dentro de um contexto de persistência (ligação à BD).
- **EntityTransaction** - representa uma transação na DB que faz o conjunto de alterações do entity manager.
 - Delimitada por `.begin()` e `.commit()`
 - Se algo correr mal (RollbackException), chamar `.rollback()`

EntityManager

```
public interface CustomUserRepository {  
    User customFindMethod(Long id);  
}
```

```
public class CustomUserRepositoryImpl implements CustomUserRepository {
```

```
@PersistenceContext
```

```
private EntityManager entityManager;
```

○ Spring preenche este atributo automaticamente



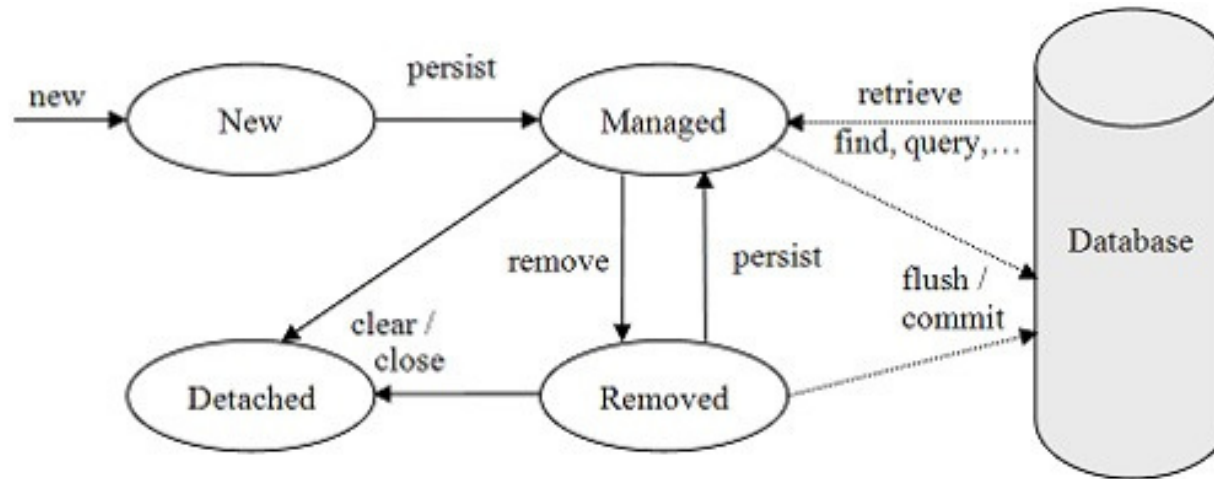
```
@Override
```

```
public User customFindMethod(Long id) {
```

```
    return (User) entityManager.createQuery("FROM User u WHERE u.id = :id")  
        .setParameter("id", id)  
        .getSingleResult();
```

```
    }  
}
```

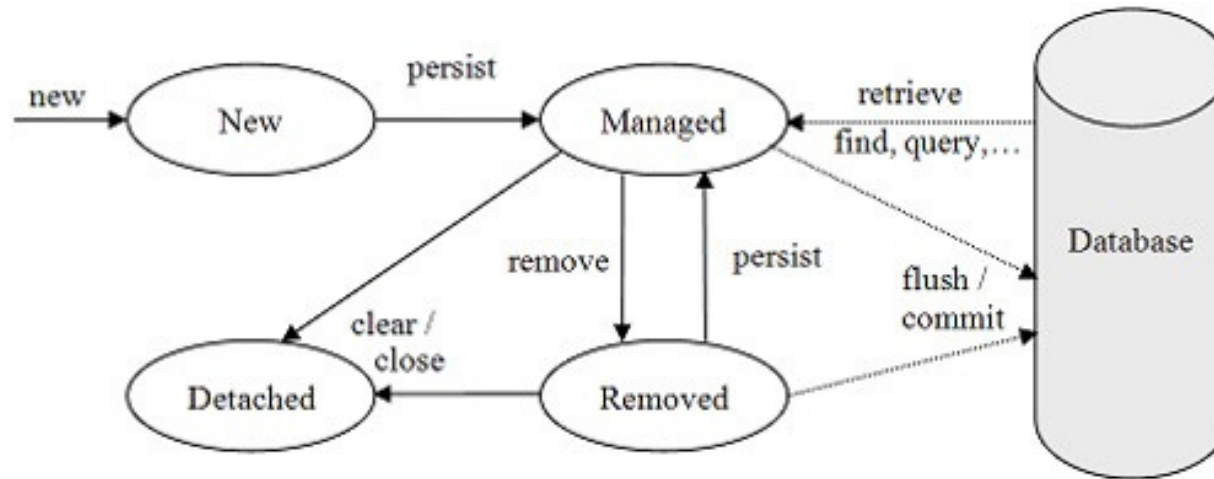
Ciclo de Vida de Entidades



```
zacarias = em.find(Student.class, 46123)
```

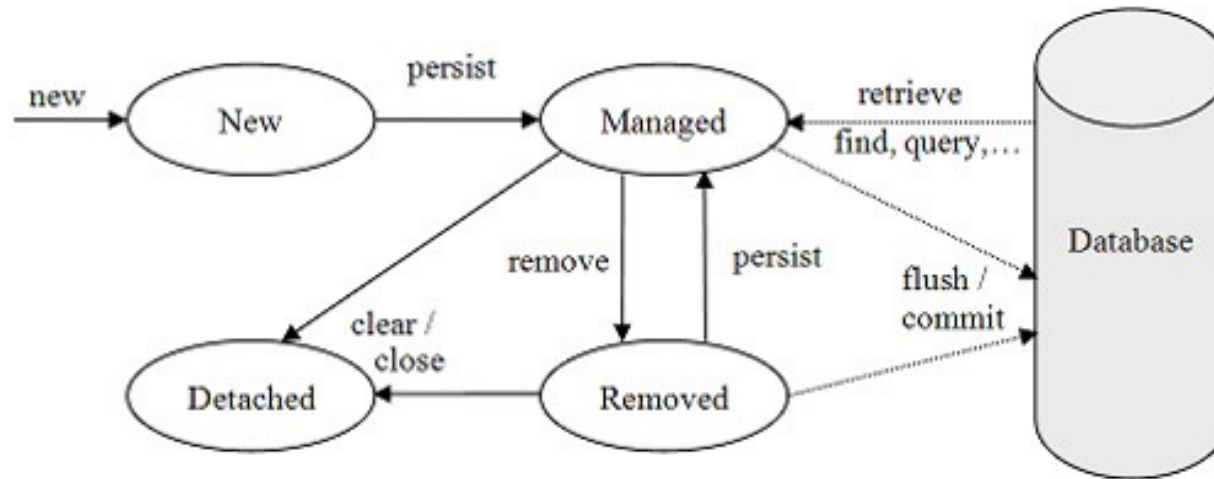
procura por PK, e cria uma instância managed.

Ciclo de Vida de Entidades



em `em.persist(josefina)`
marca o objecto para ser criado na BD.

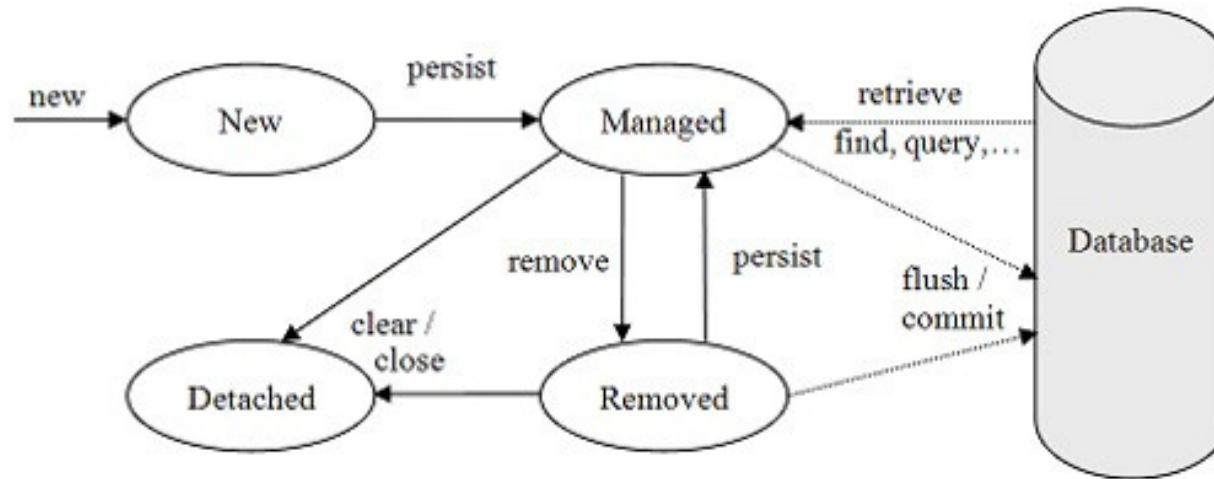
Ciclo de Vida de Entidades



em `remove(josefina)`

marca o objecto para ser apagado na BD.

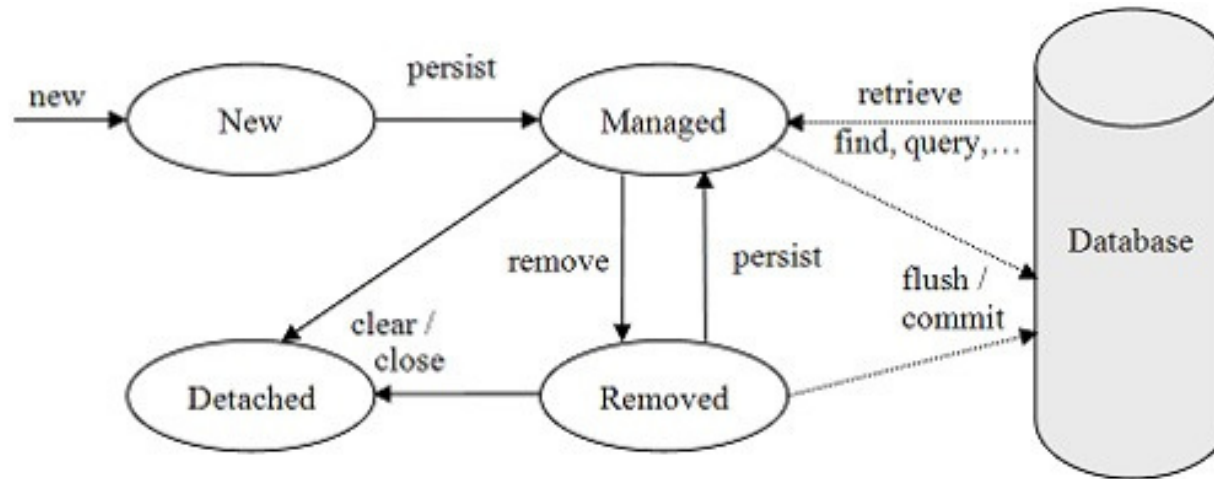
Ciclo de Vida de Entidades



```
josefina2 = em.merge(josefina)
```

devolve uma cópia managed (josefina2) com o estado da cópia antiga (josefina), que fica unmanaged.

Ciclo de Vida de Entidades



`zacarias = em.find(Student.class, 46123)`
procura por PK, e cria uma instância managed.

`em.persist(josefina)`
marca o objecto para ser criado na BD.

`em.remove(josefina)`
marca o objecto para ser apagado na BD.

`josefina2 = em.merge(josefina)`
devolve uma cópia managed (`josefina2`) com o estado da cópia antiga (`josefina`), que fica unmanaged.

Qual o nome final do Serafim?

```
// Primeira transacção
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
Person u = new Person(name:"Serafim");
em.persist(u);
em.getTransaction().commit();
```

```
// Segunda transacção
EntityManager em2 = emf.createEntityManager();
em2.getTransaction().begin();
u.setName(name:"Serafimzinho");
em2.getTransaction().commit();
```

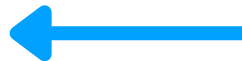
Qual o nome final do Serafim?

```
// Primeira transacção
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
Person u = new Person(name:"Serafim");
em.persist(u);
em.getTransaction().commit();
```



O objecto u foi colocado na BD, e fica unmanaged

```
// Segunda transacção
EntityManager em2 = emf.createEntityManager();
em2.getTransaction().begin();
u.setName(name:"Serafimzinho");
em2.getTransaction().commit();
```



O objecto u não está registado no em2
(nem no em, depois do commit)



Este commit não faz nada.

**Resposta: Continua a ser
Serafim!**

Solução

```
// Primeira transacção
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
Person u = new Person(name:"Serafim");
em.persist(u);
em.getTransaction().commit();

// Segunda transacção
EntityManager em2 = emf.createEntityManager();
em2.getTransaction().begin();
u.setName(name:"Serafimzinho");
Person nu = em2.merge(u);
em2.getTransaction().commit();
```

Solução

```
// Primeira transacção
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
Person u = new Person(name:"Serafim");
em.persist(u);
em.getTransaction().commit();
```

```
// Segunda transacção
EntityManager em2 = emf.createEntityManager();
em2.getTransaction().begin();
u.setName(name:"Serafimzinho");
Person nu = em2.merge(u);
em2.getTransaction().commit();
```

← O em2 passa a saber que existe o nu,
com a PK do Serafim e um novo
nome.



Este commit sabe que tem
de colocar as alterações na
linha com a PK do Serafim.

**Agora passa a ser o
Serafimzinho na BD!**

Persistência Transitiva

- O que acontece à lista de carros quando é persistido uma pessoa?
- E os seus códigos postais?
- O que acontece aos códigos postais quando a pessoa é apagada?

Persistência Transitiva

- Relações podem ter CASCADING por omissão!
- @OneToOne(cascade=CascadeType.PERSIST)
 - **Persist/Merge/Remove** - operações correspondentes
 - **Detach** - quando o dono sai do contexto, o associado também
 - **Refresh** - quando vem da BD, traz também o associado
 - **All** - todos os anteriores

Java vs JPQL

- Quando se procura informação, deve-se identificar o que queremos carregar para Java e usar o JOIN FETCH:
- `SELECT d FROM Disciplina a JOIN FETCH d.alunos`
- `d.alunos` normalmente seria carregado mais tarde (lazy loading). Com o JOIN FETCH, é carregado logo com a disciplina (1 round-trip em vez de 1+N)

Lazy Loading

- As associações de um certo objecto podem ser:
 - `fetch=FetchType.LAZY`
 - `fetch=FetchType.EAGER`
- Objectos que não estejam a ser gerido (managed), vão devolver **null** em atributos LAZY.
- Atributos LAZY são mais eficientes quando é raro serem usados.

Object-Relational Mapping

Parte 3 - Manipulação de Entidades